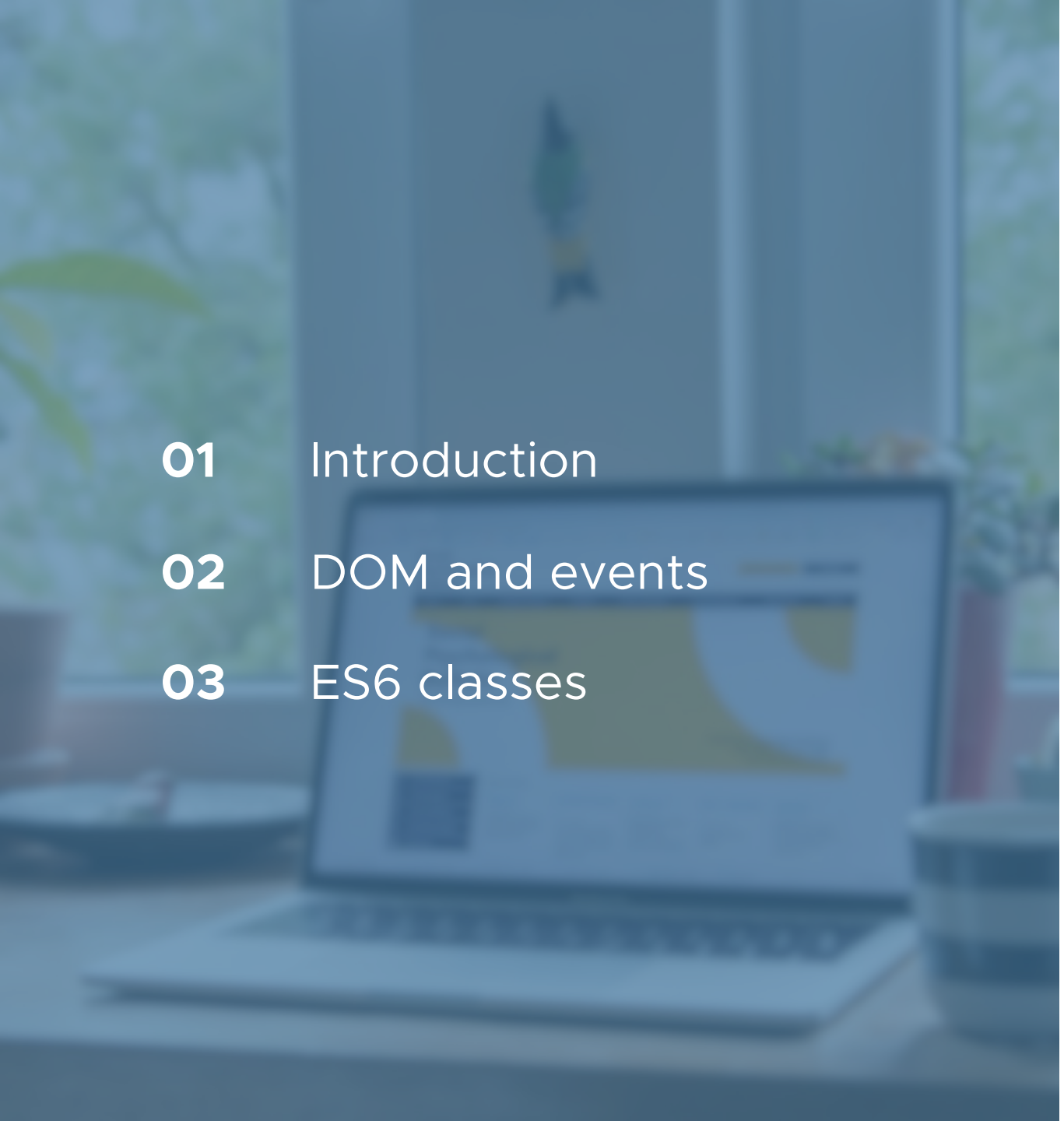
Full Stack Development Course

Eng. Elias Al Zaghrini

JavaScript

Chapter 3



Full Stack Development Course

Eng. Elias Al Zaghrini

JavaScript

Chapter 3

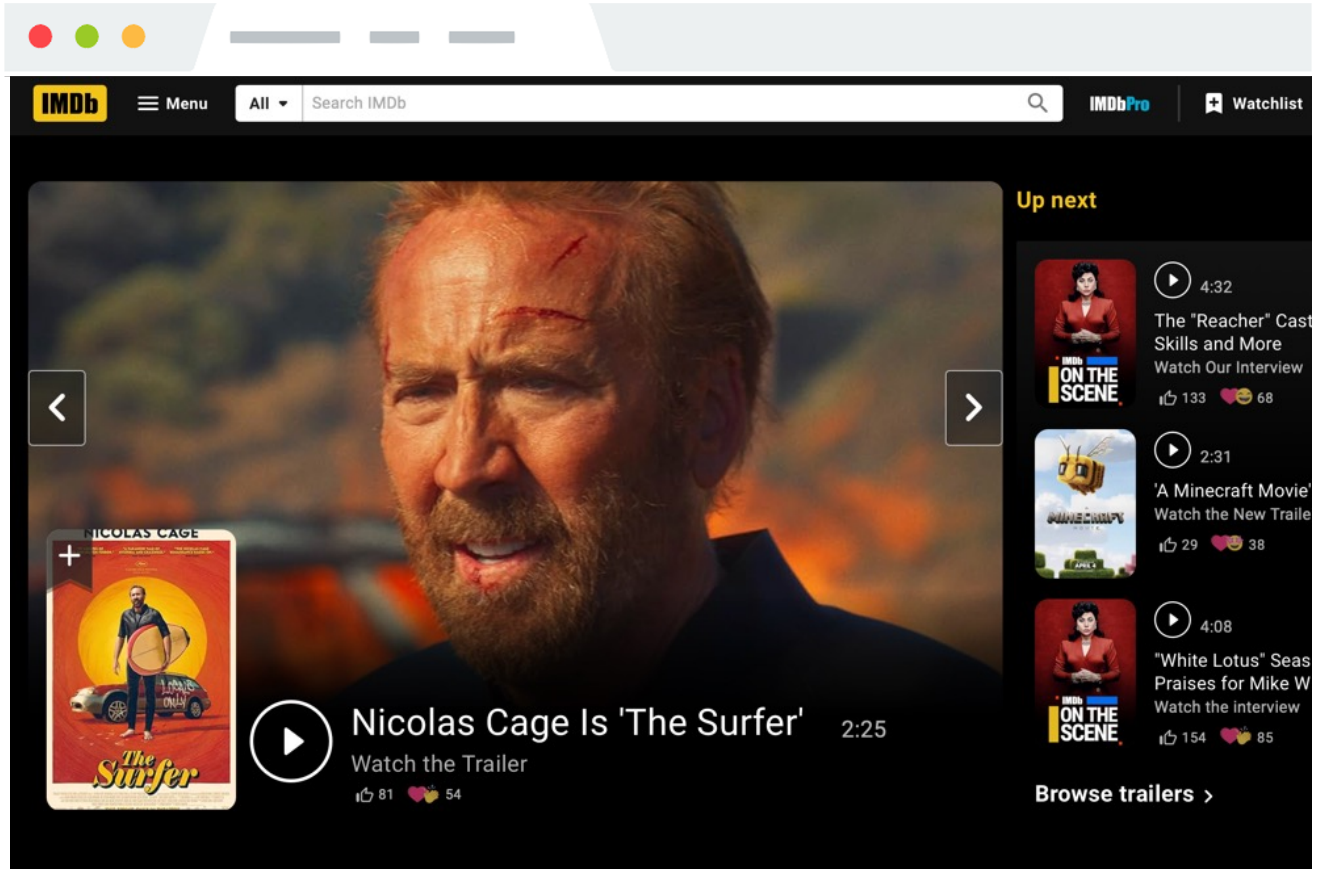
Introduction

Review

HTML + CSS =

Describes the content and structure of the page

Describes the appearance and style of the page



A web page... that doesn't do anything

Introduction

Review

We've learned how to build web pages that:

- Look the way we want them to
- Can link to other web pages
- Display differently on different screen sizes

But we don't know how build web pages that do anything:

- Get user input
- Save user input
- Show and hide elements when the user interacts with the page

Introduction

What is JavaScript?

- JavaScript is a programming language.
- It is currently the only programming language that your browser can execute natively.
- Therefore, if you want to make your web pages do stuff, you must use JavaScript: There are no other options.
- JavaScript has nothing to do with Java (named that way for marketing reasons)

Introduction

What is JavaScript?

Amateur JavaScript

- Simple, old-school JavaScript
    - Mostly not best practice
    - Everything in global scope
    - No classes / modules
    - Will result in a big mess if you code this way for anything but very small projects
- Easy to get started

Introduction

Linking JavaScript in HTML

HTML can embed JavaScript files into the web page via the `<script>` tag.

```
<!DOCTYPE html>
<html lang=“en”>
  <head>
      <title>ULFG II</title>
      <link rel="stylesheet" href="style.css" />
      <script src="filename.js"></script>
  </head>

  <body>
      ... contents of the page...
  </body>
</html>
```

Introduction

Logging

You can print log messages in JavaScript by calling `console.log()`:

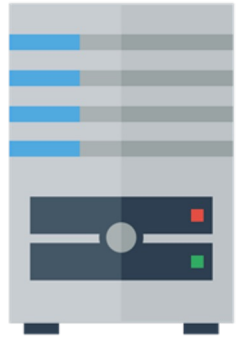
```
console.log('Hello, world!');
```

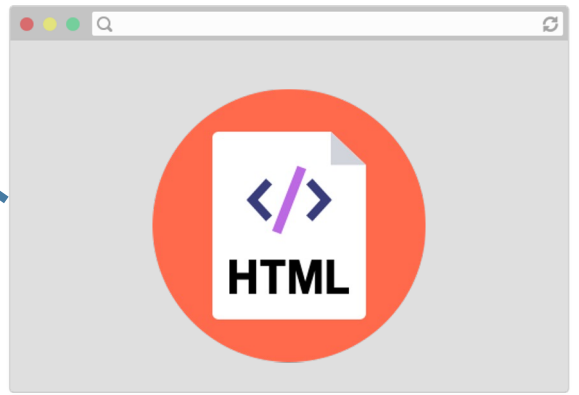
This JavaScript's equivalent of Java's `System.out.println`, `print`, `printf`, etc.

Introduction

How does JavaScript get loaded?

```
<head>
    <title>ULFG II</title>
    <link rel="stylesheet" href="style.css" /
    <script src="filename.js"></script>
</head>
```



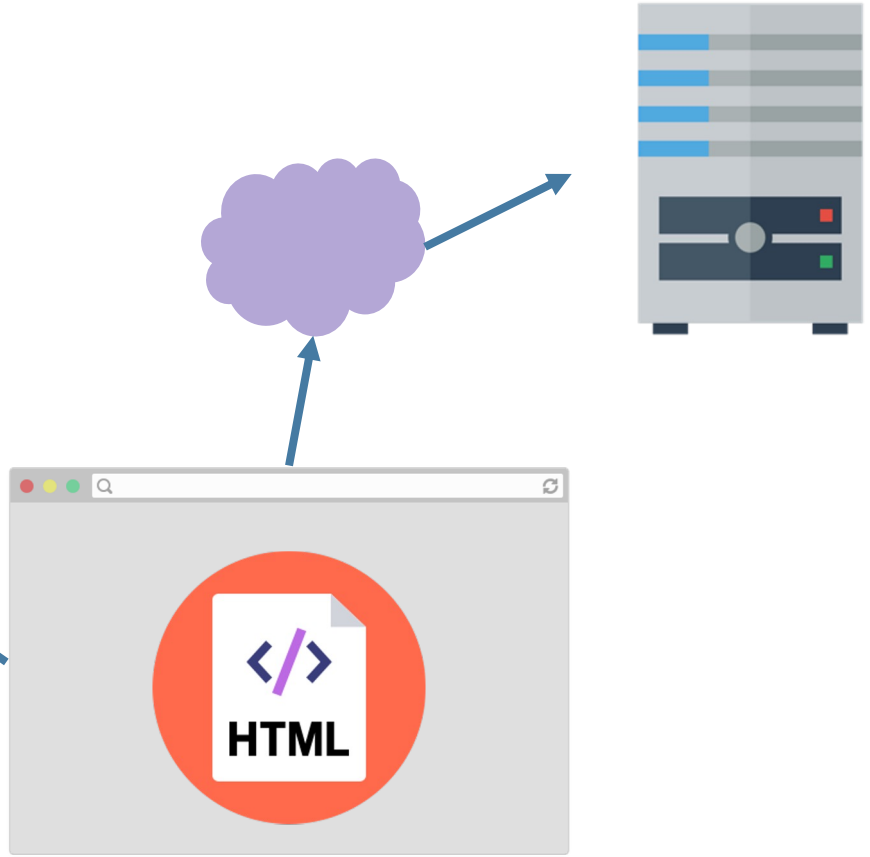


The browser is parsing the HTML file, and gets to a script tag, so it knows it needs to get the script file as well.

Introduction

How does JavaScript get loaded?

```
<head>
    <title>ULFG II</title>
    <link rel="stylesheet" href="style.css" />
    <script src="filename.js"></script>
</head>
```

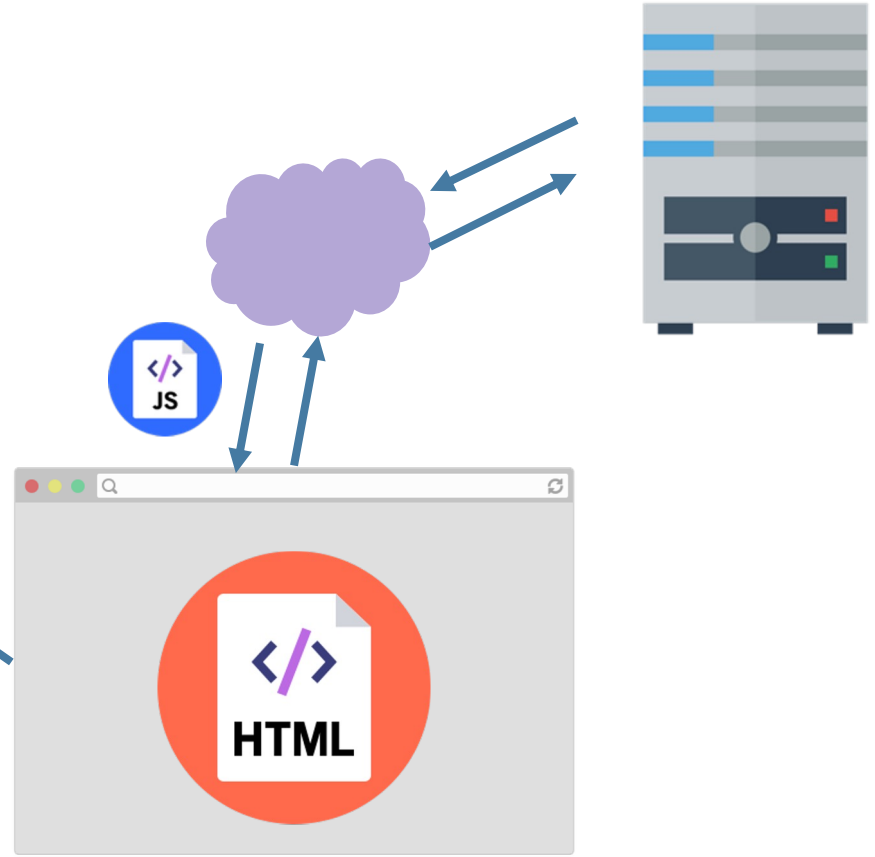


The browser makes a request to the server for the script.js file, just like it would for a CSS file or an image...

Introduction

How does JavaScript get loaded?

```
<head>
    <title>ULFG II</title>
    <link rel="stylesheet" href="style.css" /
    <script src="filename.js"></script>
</head>
```



And the server responds with the JavaScript file, just like it would with a CSS file or an image...

Introduction

How does JavaScript get loaded?

```
<head>
    <title>ULFG II</title>
    <link rel="stylesheet" href="style.css" /
    <script src="filename.js"></script>
</head>
```


```
console.log('Hello, world!');
```

Now at this point, the JavaScript file will execute "client-side", or in the browser on the user's computer.

Introduction

JavaScript execution

There is no "main method”

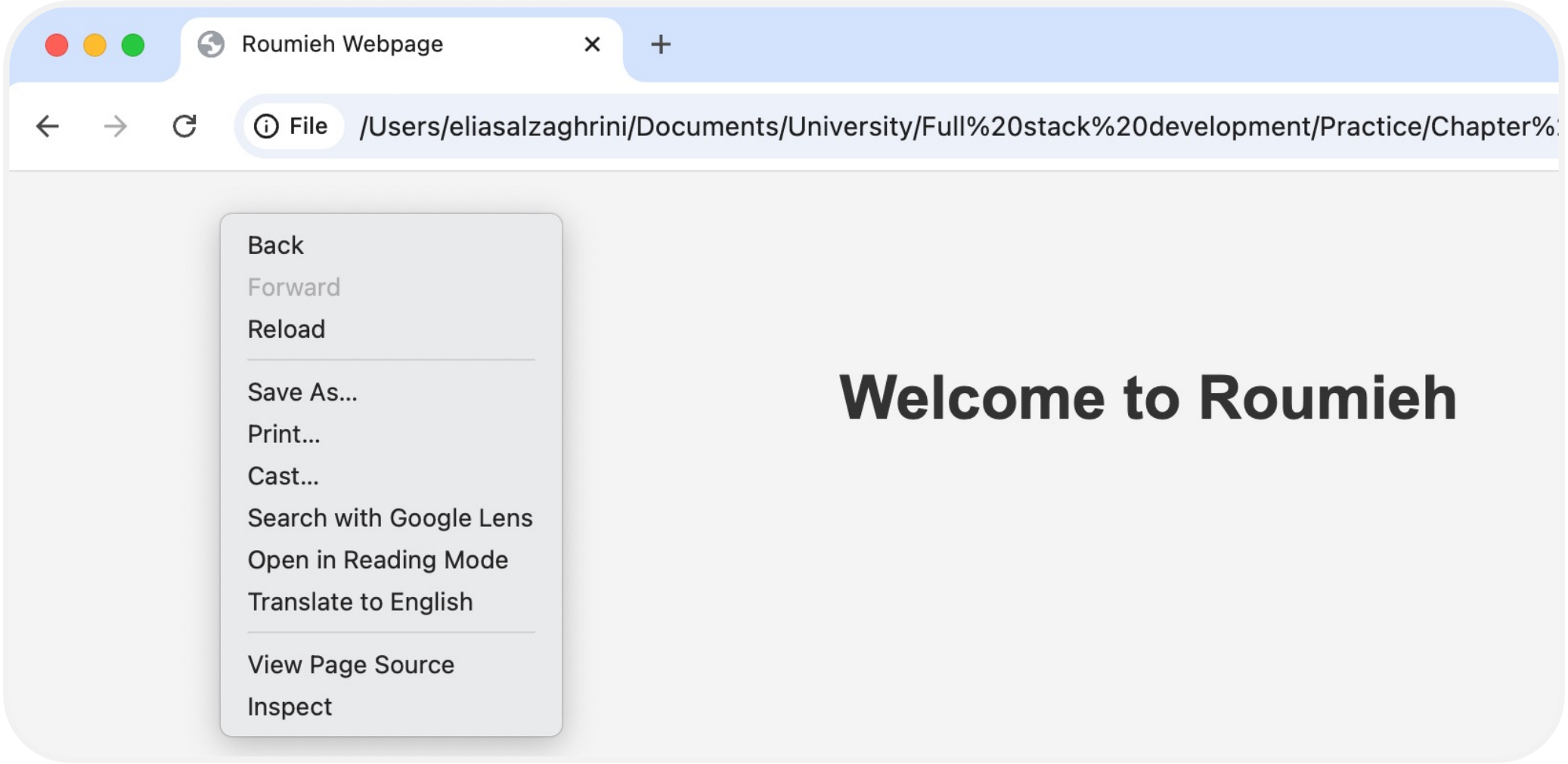
The script file is executed from top to bottom.

There's no compilation by the developer

JavaScript is compiled and executed on the fly by the browser

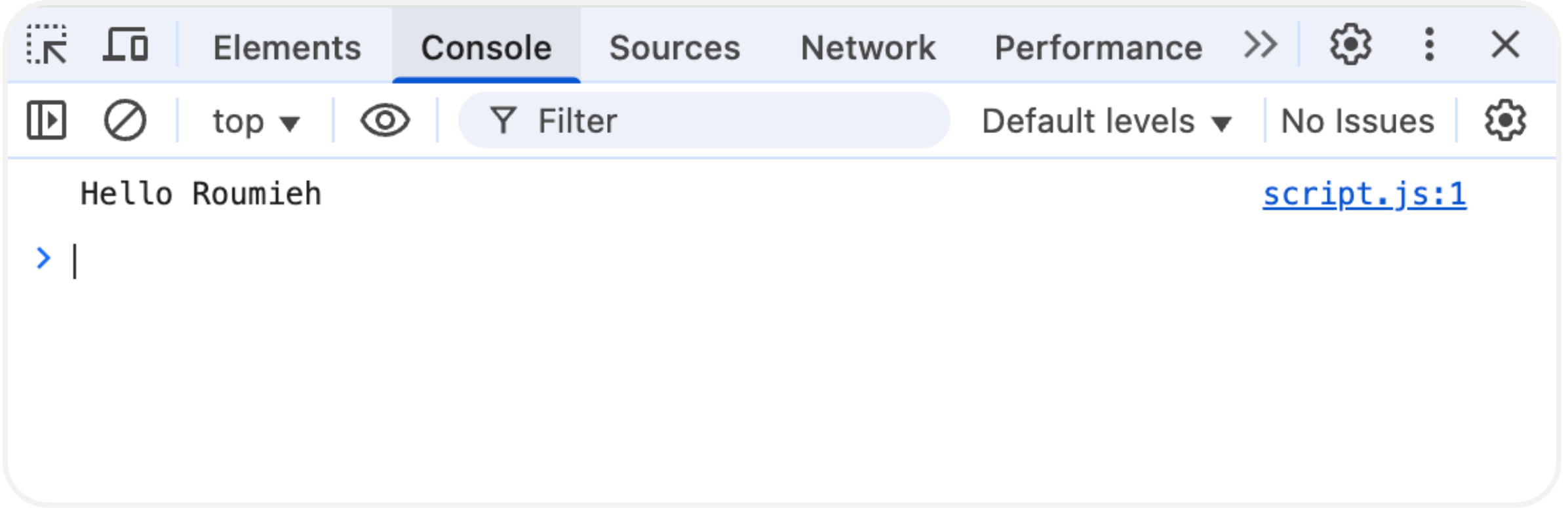
Introduction

JavaScript execution



Introduction

JavaScript execution



JavaScript

Practice

Intro

Hello Roumieh

Introduction

Syntax

- For-loops

```
for (let i = 0; i < 5; i++) { … }
```

- While-loops

```
while (notFinished) { … }
```

- Comments

```
// comment or /* comment */
```

Introduction

Syntax

- While-loops

```
if (...) {
      ...
} else {
      ...
}
```

Introduction

Syntax

- Functions

```
function name() {
  statement;
  statement;
 ...
}
```

Introduction

Syntax

- Variables and constants: Declare a variable in JS with one of three keywords

var

```
var x = 10;
var x = 20; // Allowed
x = 30;     // Allowed
```

let

```
let y = 10;
let y = 20; // Not allowed
y = 30; // Allowed
```

const

```
const z = 10;
z = 20; // Not allowed
const z; // Not allowed
```

Introduction

Syntax

- Variables and constants:
    - Use `const` whenever possible.
    - If you need a variable to be reassignable, use `let`.
    - Don't use `var`.
        - You will see many example code on the internet with var since const and let are relatively new.
        - However, const and let are well-supported, so there's no reason not to use them.

Introduction

Syntax

- Functions parameters

```
function printMessage(message, times) {
  for (var i = 0; i < times; i++) {
    console.log(message);
  }
}
```

Function parameters are **not** declared with `var`, `let`, or `const`

Introduction

Syntax

- Data types
    - There are five primitive types (MDN):
        - Boolean : true and false
        - Number : everything is a double (no integers)
        - String: in 'single' or "double-quotes”
        - Null: null: a value meaning "this has no value"
        - Undefined: the value of a variable with no value assigned
    - There are also Object types, including Array, Date, etc.

Introduction

Syntax

- Data types
    - Numbers:
        - All numbers are floating point real numbers. No integer type.
        - Operators are like Java or C++.
        - A few special values: NaN (not-a-number), +Infinity, -Infinity
        - There's a Math class: Math.floor, Math.ceil, etc.

```
const midterm = 0.2;

const final = 0.35;

const score = mid * 90 + final * 95;

console.log(score);  // 90.4
```

Introduction

Syntax

- Data types
  - Strings:
    - Can be defined with single or double quotes
    - No char type: letters are strings of length one
    - Can use plus for concatenation
    - Can check size via length property (not function)

```
let snack = 'coo';

snack += 'kies';

snack = snack.toUpperCase();

console.log("I want " + snack);
```

Introduction

Syntax

- Data types
  - Boolean:
    - There are two literal values for boolean: true and false that behave as you would expect
    - Can use the usual boolean operators: && || !

```
let isHungry = true;
let isTeen = age > 12 && age < 20;

if (isHungry && isTeen) {
  pizza++;
}
```

Introduction

Syntax

- Data types
    - Boolean:
        - JavaScript's == and != are broken: *they do an implicit type conversion before the comparison*.
        - Instead of fixing == and != , the ECMAScript standard kept the existing behavior but added === and !==
        - Always use === and !== and don't use == or !=

Introduction

Syntax

- Data types
    - Boolean:

```
'' == '0'  // false
'' == 0  // true
0 == '0'  // true
NaN == NaN  // false
[''] == ''  // true
false == undefined  // false
false == null  // false
null == undefined  // true
```

```
'' === '0'  //false
'' === 0  //false
0 === '0'  //false
NaN == NaN  //still weirdly false
[''] === ''  //false
false === undefined  //false
false === null  //false
null === undefined  //false
```

Introduction

Syntax

- Data types
    - Null and Undefined:
        - `null` is a value representing the absence of a value.
        - `undefined` is the value given to a variable that has not been a value.

```
let x = null;
let y;
console.log(x);
console.log(y);
```

```
null
undefined
>
```

Introduction

Syntax

- Data types
  - Arrays:
    - Arrays are Object types used to create lists of data.
    - 0-based indexing
    - Mutable
    - Can check size via length property (not function)

```
// Creates an empty list

let list = [];

let groceries = ['milk', 'cocoa
puffs’];

groceries[1] = 'kix';
```

Introduction

Syntax

- Data types
    - Arrays:
        - Looping through an array with the familiar for-loop :

```
let groceries = ['milk', 'cocoa puffs', 'tea'];

for (let i = 0; i < groceries.length; i++) {
  console.log(groceries[i]);
}
```

Introduction

Syntax

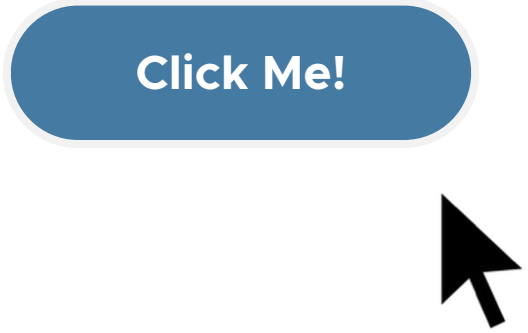
- Data types
    - Arrays:
        - Looping through an array via `for...of` :

```
for (let item of groceries) {
    console.log(item);
}
```

DOM and events

Event-driven programming

Most JavaScript written in the browser is **event-driven**: The code doesn't run right away, but it executes after some event fires.



Example:
Here is a UI element that the user can interact with.

DOM and events

Event-driven programming

Most JavaScript written in the browser is **event-driven**: The code doesn't run right away, but it executes after some event fires.

When the user clicks the button...

DOM and events

Event-driven programming

Most JavaScript written in the browser is **event-driven**: The code doesn't run right away, but it executes after some event fires.

...the button emits an "**event**," which is like an announcement that some interesting thing has occurred.

DOM and events

Event-driven programming

Most JavaScript written in the browser is **event-driven**: The code doesn't run right away, but it executes after some event fires.


```
function onClick() {
  ...
}
```

Any function listening to that event now executes. This function is called an "event handler."

DOM and events

Event-driven programming

Events can be called by almost every HTML element, especially by:

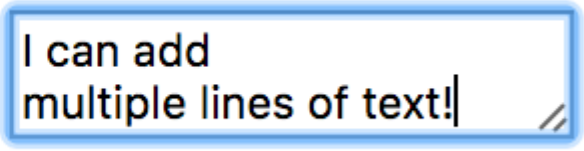
- Buttons

```
<button>Click me</button>
```

- Single-line / multi-line text input

```
<input type="text" />
```


```
<textarea></textarea>
```



DOM and events

Event-driven programming

Let's print "Clicked" to the Web Console when the user clicks the given button:

```
<button>Click me</button>
```

How to do it?

We need to add an event listener to the button...

How do we talk to an element in HTML from JavaScript?

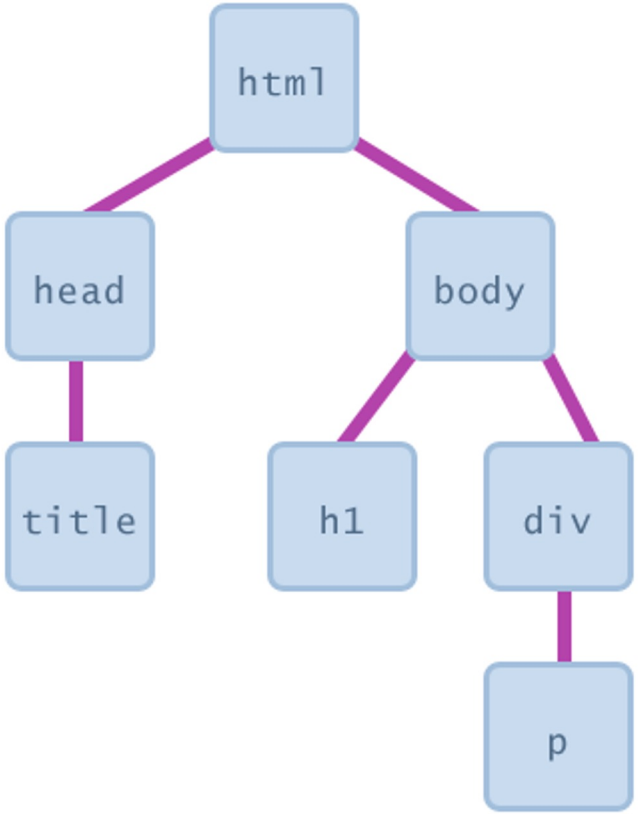
Using the DOM

DOM and events

DOM definition

Every element on a page is accessible in JavaScript through the **DOM**: **Document Object Model**

- The DOM is the tree of nodes corresponding to HTML elements on a page.
- Can modify, add, and remove nodes on the DOM, which will modify, add, or remove the corresponding element on the page.



DOM and events

Getting DOM objects

we can access any HTML element’s corresponding DOM object using the document.querySelector() function by passing a valid CSS selector such as a tag name (e.g., 'p'), class (e.g., '.my-class'), or ID (e.g., '#my-id'). :

```
document.querySelector('css selector’);
```

This returns the **first** element that matches the given CSS selector

```
// Returns the element with id="button"
let element = document.querySelector('#button');
```

DOM and events

Getting DOM objects

And via the `querySelectorAll` function, which returns **all** elements that match the given CSS selector:

```
document.querySelectorAll('css selector');
```

DOM and events

Adding event listeners

Each DOM object has the following function:

```
addEventListener(event name, function name);
```

- **Event name** is the string name of the JavaScript event you want to listen to (click, focus, blur, etc)
- **Function name** is the name of the JavaScript function you want to execute when the event fires

DOM and events

Adding event listeners

Roughly every **attribute** on an HTML element is a **property** on its respective DOM object...

```
<img src="puppy.png" />
```

```
const element = document.querySelector('img');
element.src = 'bear.png';
```

DOM and events

Troubleshooting

```
<html>
▼<head>
    <meta charset="utf-8">
    <title>First JS Example</title>
    <script src="script.js"></script>
  </head>
▼<body>
    <button>Click Me!</button>
  </body>
</html>
```

DOM and events

Troubleshooting

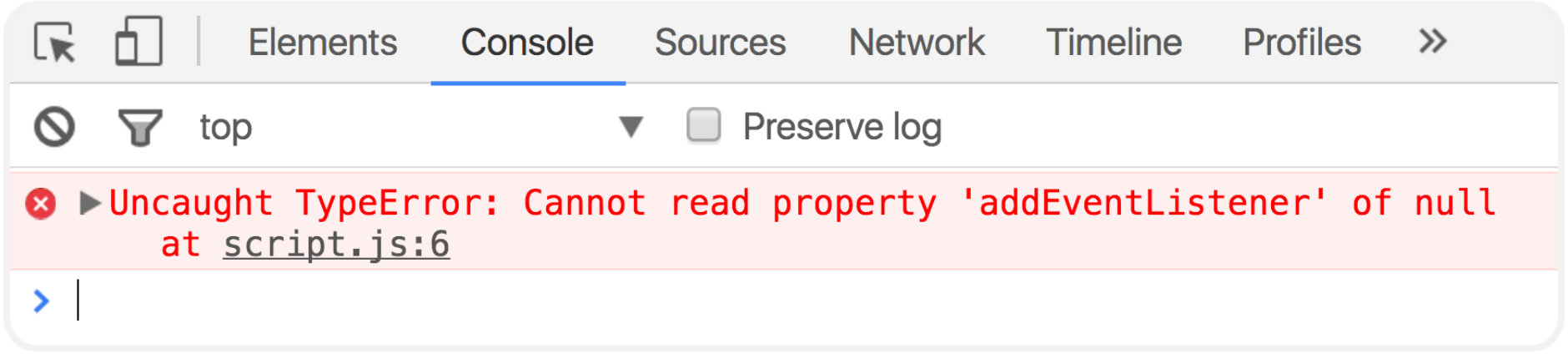
```
function onClick() {
  console.log('clicked');
}

const button = document.querySelector('button');
button.addEventListener('click', onClick);
```

DOM and events

Troubleshooting

script.js

```
1  function onClick() {
2    console.log('clicked');
3  }
4
5  const button = document.querySelector('button');
6  button.addEventListener('click', onClick);
7
```



DOM and events

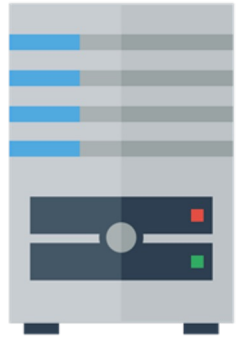
Troubleshooting

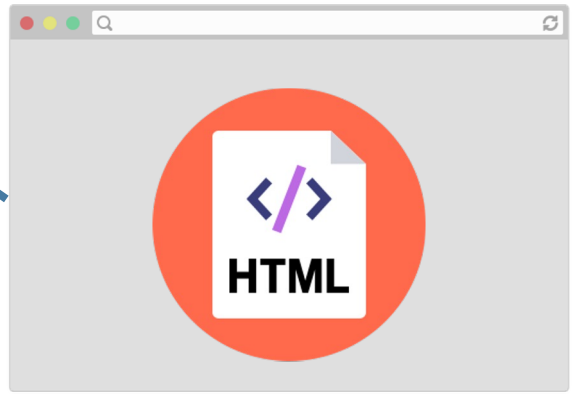
Why this error?

DOM and events

Troubleshooting

```
<head>
    <title>ULFG II</title>
    <link rel="stylesheet" href="style.css" /
    <script src=”script.js"></script>
</head>
```

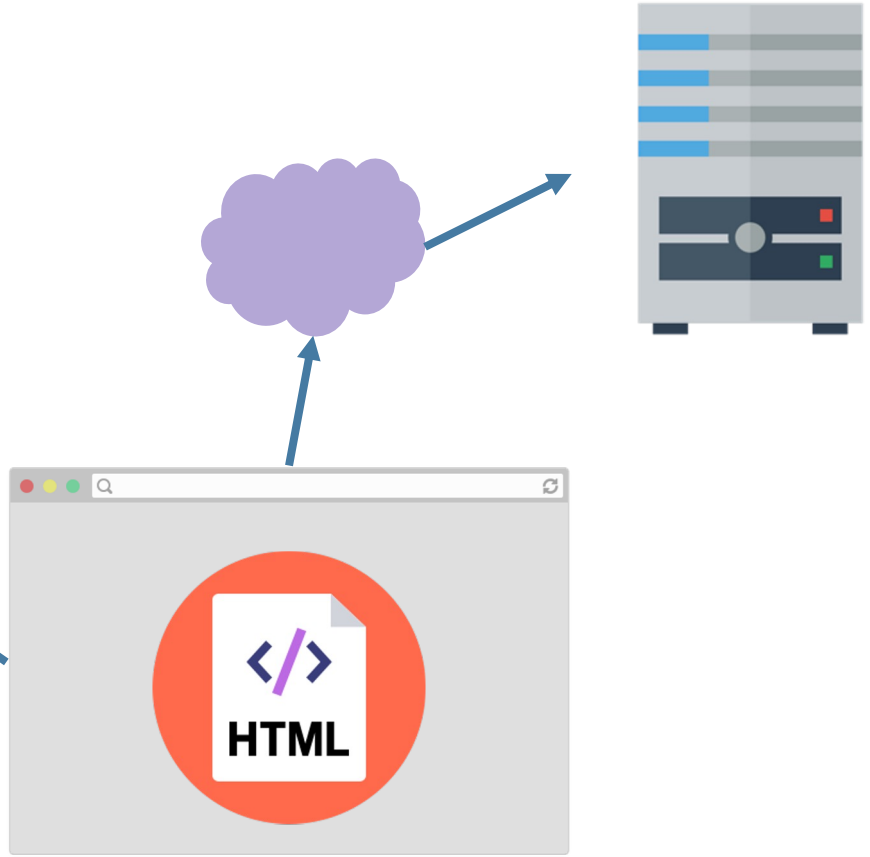




DOM and events

Troubleshooting

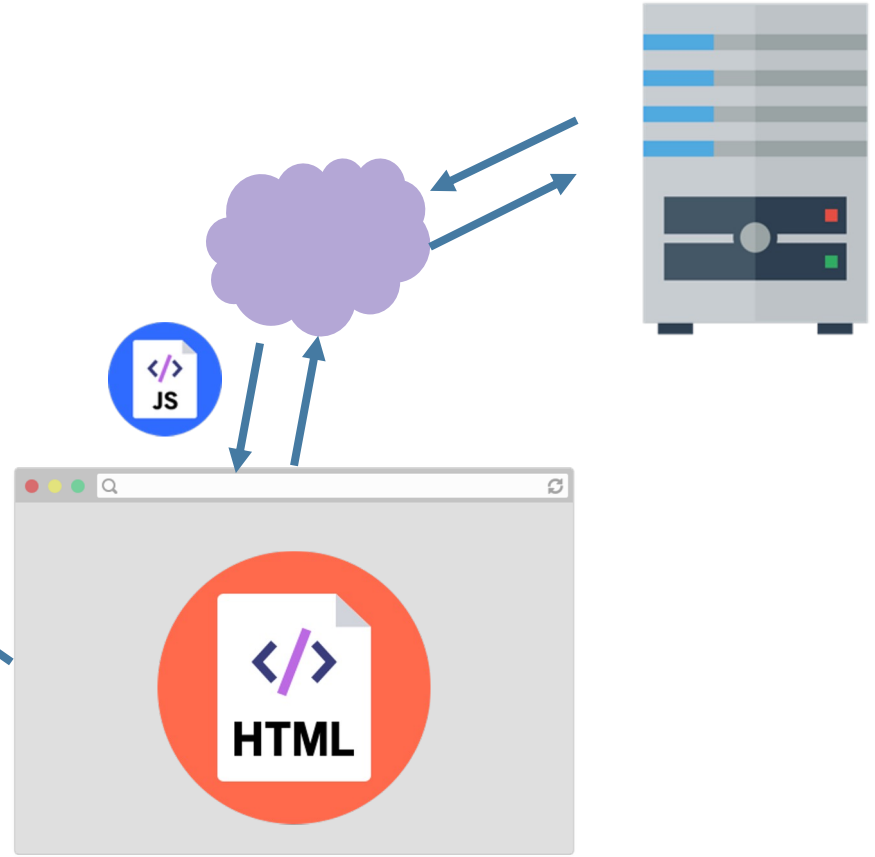
```
<head>
    <title>ULFG II</title>
    <link rel="stylesheet" href="style.css" /
    <script src=" script.js"></script>
</head>
```



DOM and events

Troubleshooting

```
<head>
    <title>ULFG II</title>
    <link rel="stylesheet" href="style.css" />
    <script src="script.js"></script>
</head>
```



DOM and events

Troubleshooting

```
<head>
    <title>ULFG II</title>
    <link rel="stylesheet" href="style.css" /
    <script src=”script.js"></script>
</head>
```


```
function onClick() {
  console.log('clicked');
}

const button = document.querySelector('button');
button.addEventListener('click', onClick);
```

DOM and events

Troubleshooting

```
<head>
    <title>ULFG II</title>
    <link rel="stylesheet" href="style.css" /
    <script src=”script.js"></script>
</head>
```


```
function onClick() {
  console.log('clicked');
}

const button = document.querySelector('button');
button.addEventListener('click', onClick);
```

DOM and events

Troubleshooting

```
<head>
    <title>ULFG II</title>
    <link rel="stylesheet" href="style.css" /
    <script src=”script.js"></script>
</head>
```


```
function onClick() {
  console.log('clicked');
}

const button = document.querySelector('button');
button.addEventListener('click', onClick);
```

DOM and events

Troubleshooting

- We are only at the <script> tag, which is at the top of the document... so the <button> isn't available yet.
- Therefore, `querySelector` returns null, and we can't call `addEventListener` on null.
- You can add the `defer` attribute to the script tag so that the JavaScript doesn't execute until after the DOM is loaded

```
<script src="script.js" defer></script>
```

- Other old-school ways of doing this (don't do these):
    - Put the `<script>` tag at the bottom of the page
    - Listen for the "load" event on the window object

DOM and events

Troubleshooting

Log messages aren't so interesting...

How do we interact with the page?

DOM and events

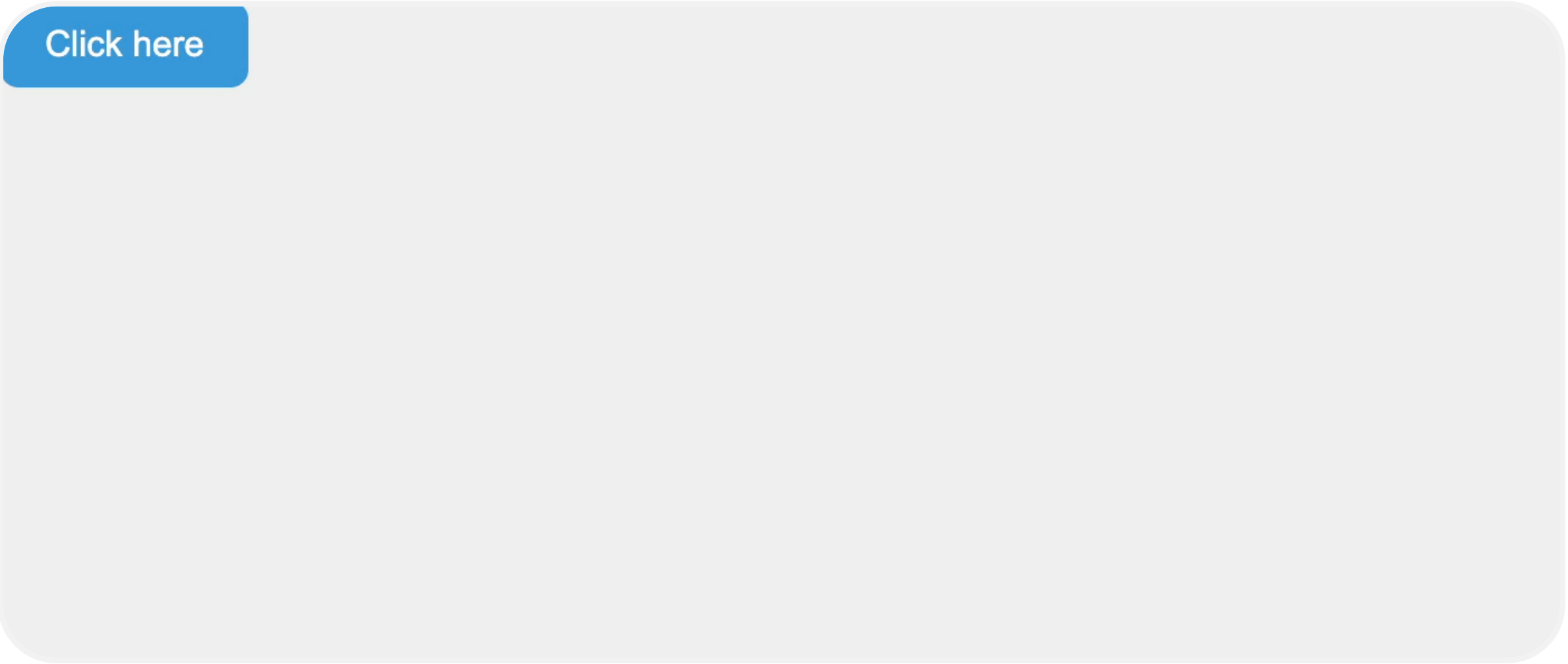
Practice

Challenge – 15 minutes

The tricky button

DOM and events

Practice



DOM and events

Maps through Objects

- Every JavaScript object is a collection of property-value pairs.
- Therefore, you can define maps by creating Objects:

```
// Creates an empty object
const prices = {};
const scores = {
  'peach': 100,
  'mario': 88,
  'luigi': 91
};
console.log(scores['peach']);   // 100
```

DOM and events

Maps through Objects

- String keys do not need quotes around them. Without the quotes, the keys are still of type string.

```
// Creates an empty object
const prices = {};
const scores = {
  peach: 100,
  mario: 88,
  luigi: 91
};
console.log(scores['peach']);   // 100
```

DOM and events

Maps through Objects

- There are two ways to access the value of a property:
    1. `objectName[property]`
    2. `objectName.property` (Only works for String keys)

```
const scores = {
  peach: 100,
  mario: 88,
  luigi: 91
};
console.log(scores['peach']);   // 100
console.log(scores.luigi);   // 91
```

DOM and events

Maps through Objects

- To add a property to an object, name the property and give it a value:

```
const scores = {
  peach: 100,
  mario: 88,
  luigi: 91
};
scores.toad = 72;
let name = 'wario';
scores[name] = 102;
console.log(scores);
```

```
▶ Object {peach: 100, mario: 88, luigi: 91, toad: 72, wario: 102}
```

DOM and events

Maps through Objects

- To remove a property to an object, use delete:

```
const scores = {
  peach: 100,
  mario: 88,
  luigi: 91
};
scores.toad = 72;
let name = 'wario';
scores[name] = 102;
delete scores.peach;
console.log(scores);
```

```
▶ Object {mario: 88, luigi: 91, toad: 72, wario: 102}
```

DOM and events

Current target

- The event's `currentTarget` property is a reference to the object that we attached to the event, in this case the <img>'s `Element` to which we added the listener.
- An `Event` element is passed to the listener as a parameter

```
function openPresent(event) {
    const image = event.currentTarget;
    image.src = 'https://media.giphy.com/media/27ppQUOxe7K1G/giphy.gif';}

const image = document.querySelector('img');
image.addEventListener('click', openPresent);
```

DOM and events

Some properties of Element objects

Property	Description
`id`	The value of the id attribute of the element, as a string
`innerHTML`	The raw HTML between the starting and ending tags of an element, as a string
`textContent`	The text content of a node and its descendants.
`classList`	An object containing the classes applied to the element

DOM and events

Some properties of Element objects

- We can select the h1 element, then set its textContent to change what is displayed in the h1.

```
function openPresent(event) {
    const image = event.currentTarget;
    image.src = 'https://media.giphy.com/media/27ppQUOxe7K1G/giphy.gif';

    const title = document.querySelector('h1');
    title.textContent = 'Hooray!';}

const image = document.querySelector('img');
image.addEventListener('click', openPresent);
```

DOM and events

Class list

Add a class

- `classList` is a property of DOM elements.
- Adds one or more class names to an element.
- Does not duplicate classes if they already exist.

```
let box = document.querySelector(".box");
box.classList.add("highlight");
```

Result:

```
<div class="box highlight"></div>
```

DOM and events

Class list

Removing a class

- Removes one or more class names from an element. If the class doesn’t exist, nothing happens.

```
let box = document.querySelector(".box");
box.classList.remove("highlight");
```

Before:

```
<div class="box highlight"></div>
```

After:

```
<div class="box"></div>
```

DOM and events

Class list

Real-World Example: Toggle Highlight on Click

- Removes one or more class names from an element. If the class doesn’t exist, nothing happens.

```
<button id="toggleBtn">Toggle Highlight</button>
<div id="myBox" class="box"></div>
```

DOM and events

Class list

Real-World Example: Toggle Highlight on Click

- Removes one or more class names from an element. If the class doesn’t exist, nothing happens.

```
const button = document.getElementById("toggleBtn");
const box = document.getElementById("myBox");

button.addEventListener("click", () => {
  box.classList.contains("highlight")
    ? box.classList.remove("highlight")
    : box.classList.add("highlight");
});
```

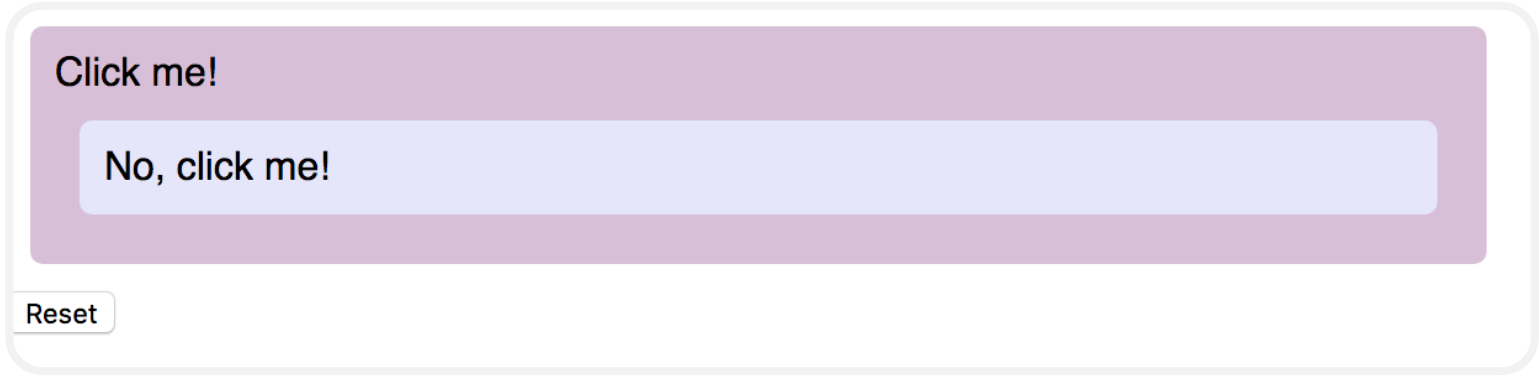
DOM and events

Multiple event listeners

What if you have event listeners set on both an element and a child of that element?

- Do both fire?
- Which fires first?

```
<div id="outer">
  Click me!
  <div id="inner">
    No, click me!
  </div>
</div>
```



DOM and events

Multiple event listeners

What if you have event listeners set on both an element and a child of that element?

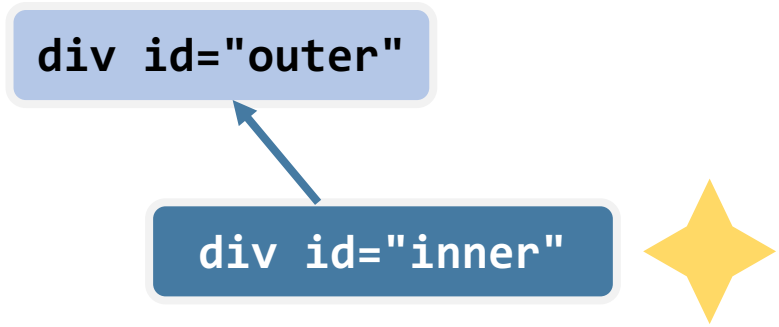
- Do both fire?
- Which fires first?

```
const outer = document.querySelector('#outer');
const inner = document.querySelector('#inner');
outer.addEventListener('click', onOuterClick);
inner.addEventListener('click', onInnerClick);
```

DOM and events

Multiple event listeners

- Both events fire if you click the inner element
- By default, the event listener on the inner-most element fires first




```
<div id="outer">
  Click me!
  <div id="inner">
    No, click me!
  </div>
</div>
```

- This event ordering (inner-most to outer-most) is known as **bubbling**.

DOM and events

Multiple event listeners

We can stop the event from bubbling up the chain of ancestors by using **`event.stopPropagation()`**:

```
function onInnerClick(event) {
  inner.classList.add('selected');
  console.log('Inner clicked!');
  event.stopPropagation();
}
```

DOM and events

Keyboard events

Event name	Description
`keydown`	Fires when any key is pressed. Continues firing if you hold down the key
`keypress`	Fires when any character key is pressed, such as a letter or number. Continues firing if you hold down the key.
`keyup`	Fires when you stop pressing a key.

You can listen for keyboard events by adding the event listener to `document`:

```
document.addEventListener('keyup', onKeyUp);
```

DOM and events

Example

Example

Key events

DOM and events

Mouse events

Event name	Description
`click`	Fired when you click and release
`mousedown`	Fired when you click down
`mouseup`	Fired when when you release from clicking
`mousemove`	Fired repeatedly as your mouse moves

*`mousemove` only works on desktop, since there's no concept of a mouse on mobile.

DOM and events

Touch events

Event name	Description
`touchstart`	Fired when you touch the screen
`touchend`	Fired when you lift your finger off the screen
`touchmove`	Fired repeatedly while you drag your finger on the screen
`touchcancel`	Fired when a touch point is "disrupted" (e.g. if the browser isn't totally sure what happened)

* **`touchmove`** only works on mobile

DOM and events

Position

`MouseEvents` have a `clientX` and `clientY` :

- `clientX`: x-axis position relative to the left edge of the browser viewport
- `clientY`: y-axis position relative to the top edge of the browser viewport

```
function onClick(event) {
  console.log('x' + event.clientX);
  console.log('y' + event.clientY);
}
element.addEventListener('click', onClick);
```

ES6 Classes

Amateur JavaScript

So far, the JavaScript code we've been writing has looked like this:

- Mostly all in one file
- All global functions
- Global variables to save state between events

It would be nice to write code in a modular way...

ES6 Classes

Amateur JavaScript

Wait a minute...

Wasn't JavaScript created in 1995?

And classes were introduced... 20 years later in 2015?

Was it not possible to create classes in JavaScript before 2015?!

ES6 Classes

Amateur JavaScript

"There is one thing I am certain is a bad part, a very terribly bad part, and that is the new class syntax [in JavaScript]... The people who are using class will go to their graves never knowing how miserable they were."

Douglas Crockford, author of *JavaScript: The Good Parts*; prominent speaker on JavaScript; member of TC39

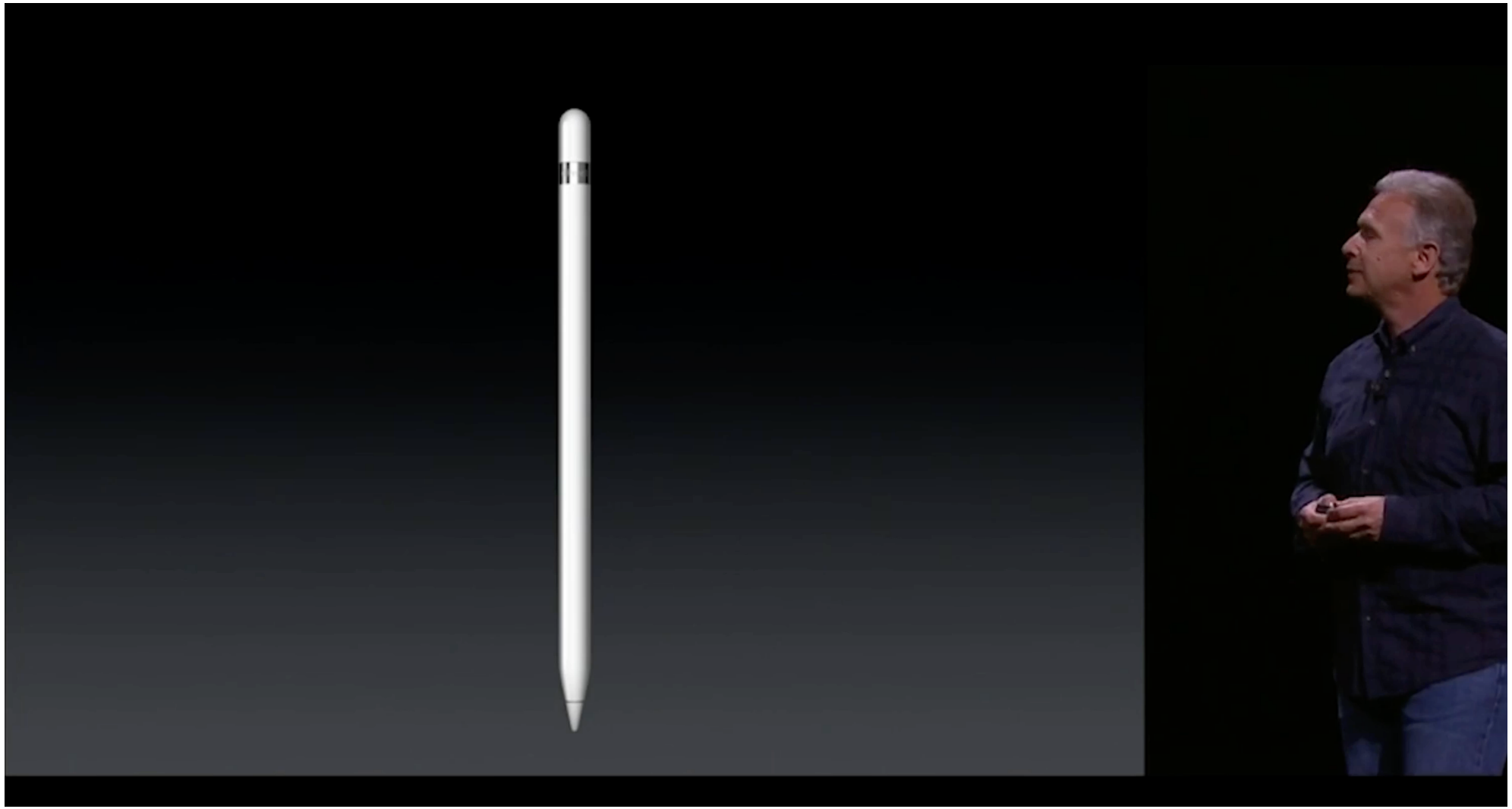
(committee that makes ES decisions)

ES6 Classes

Amateur JavaScript

ES6 Classes

Amateur JavaScript



ES6 Classes

Why classes?

- For a sufficiently small task, global variables, functions, etc., are fine
- But for a larger website, your code will be hard to understand and easy to break if you do not organize it
- Using classes and object-oriented design is the most common strategy for organizing code

ES6 Classes

Why classes?

Well-engineered software is well-organized software:

- Software engineering is all about knowing
    - What to change
    - Where to change it
- You can read an existing codebase better if it is well-organized
- "Why do I need to read a codebase?" Because you need to modify the codebase to add features and fix bugs

ES6 Classes

Why classes?

Having a bunch of loose variables in the global scope is asking for trouble

- Much easier to hack
    - Can be accessed via extension or Web Console
    - Can override behaviors
- Global scope gets polluted
    - What if you have two functions with the same name? One definition is overridden without error
- Very easy to modify the wrong state variable

ES6 Classes

Public methods

- `constructor` is optional.
- Parameters for the constructor and methods are defined in the same way as they are for global functions.
- You do not use the `function` keyword to define methods.
- Within the class, you must always refer to other methods in the class with the `this.` prefix.
- All methods are **public**, and you **cannot** specify private methods… yet.

```
class ClassName {

  constructor(params) {
    ...
  }

  methodName() {
     this.methodTwo();
  }

  methodName() {
    ...
  }
}
```

ES6 Classes

Public fields

- Define public fields by setting **`this.`*`fieldName`*** in the constructor… or in any other function.
- Within the class, you must always refer to fields with the `this.` prefix.
- You cannot define private fields

```
class ClassName {

  constructor(params) {
    this.fieldName = fieldValue;
    this.someField = someParam;
  }

  methodName() {
    const someValue = this.somefield;
  }
}
```

ES6 Classes

Instantiation

Create new objects using the new keyword

```
class SomeClass {

  ...

  someMethod() { … }
}

const x = new SomeClass();
const y = new SomeClass();
y.someMethod();
```

ES6 Classes

Example

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound.`);
  }
}

const dog = new Animal("Dog");
dog.speak(); // Output: Dog makes a sound.
```

ES6 Classes

Full example

```
<head>
  <meta charset="UTF-8">
  <title>Animal Example</title>
  <script type="module" src="main.js"></script>
</head>

<body>
  <button id="speakBtn">Make Animal Speak</button>
</body>
```

ES6 Classes

Full example

Animal.js

```
export class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound.`);
    alert(`${this.name} makes a sound.`);
  }
}
```

ES6 Classes

Full example

main.js

```
import { Animal } from './Animal.js';

const myAnimal = new Animal("Leo");

const element = document.getElementById("speakBtn")

element.addEventListener("click", () => {
  myAnimal.speak();
});
```

ES6 Classes

Practice

Practice

Drums Game

JavaScript

Any questions?